

Enabling the MessageFoundry Console on a Remote PC

Early Access · MessageFoundry v0.3.2 · September 2026

Retired UI (BACKLOG #103, 2026-07-13). The PySide6 desktop console is retired. The engine now serves the browser web console at `/ui` (ADR 0065). Use the certificate and network settings below for remote access. Then open `https://<engine-host>:8765/ui` on each PC. No desktop installation is necessary. The `messagefoundry-console --url ...` commands remain only as historical reference.

This guide helps IT administrators give staff browser access to MessageFoundry from their own PCs. Prepare a certificate, configure network access, and open the engine's `/ui` page.

No code changes are required. Everything here is configuration.

What you're setting up

The MessageFoundry **engine** runs on a server, usually as a Windows service. It serves the **web console** at `/ui` from the separate `messagefoundry-webconsole` distribution. Install that package on the server with `pip install messagefoundry-webconsole`. No desktop application is needed on remote PCs. By default, the engine accepts connections only from **the same machine** (`127.0.0.1`). Configure network access before connecting from another PC.

To use the console from another PC, you:

1. Give the engine a **TLS certificate** (so the connection is encrypted), and
2. Tell the engine to **listen on the network**, then
3. Point each remote console at the engine's `https://` address.

TLS encrypts the connection, and each user signs in with the same account used locally. Remote access starts only after you make these changes.

Before you begin — checklist - The engine server's hostname or IP address (e.g. `mefor-srv01.hospital.local`). - A TLS certificate for that hostname (see Step 1). - The MessageFoundry console installed on each remote PC. - A network path from the PCs to the engine on its API port (default **8765**) — open the firewall if needed. - A MessageFoundry user account for each person who will sign in.

Step 1 — Prepare a TLS certificate for the engine

The engine needs a certificate so traffic (including login credentials) is encrypted. Choose **one**:

Option	When to use it	Notes
Your organization's internal CA (e.g. Active Directory Certificate Services)	Recommended for most sites	Issue a server certificate for the engine's hostname. Domain-joined PCs already trust your CA, so the console needs no extra setup.
A public CA (e.g. A commercial cert)	The engine has a public DNS name	Trusted everywhere automatically.
A self-signed certificate	Small/internal setups, pilots	The retired desktop console uses <code>--cacert</code> to select the certificate (Step 3).

The certificate's **Subject Alternative Name (SAN)** must include the hostname or IP address used to connect. You need these files, either separate or combined:

- a **certificate** PEM (e.g. `engine-cert.pem`), and
- a **private key** PEM (e.g. `engine-key.pem`).

Place them somewhere the engine's service account can read, e.g. `C:\MessageFoundry\tls\`.

Step 2 — Configure the engine server

Edit the engine's configuration file, `messagefoundry.toml`, on the server. Exposure is controlled from the `[security]` section, and the certificate paths from `[api]`:

```
[security]
local_access_only = false                # reachable from off this machine
listen_address    = "0.0.0.0"           # or a specific NIC IP, e.g. "10.0.0.12"
serve_web_console = true                 # REQUIRED explicitly when exposed (see note below)
web_console_public_address = "https://engine-host:8765" # the origin the browser uses

[api]
port = 8765                             # the API port the console connects to
tls_cert_file = "C:/MessageFoundry/tls/engine-cert.pem"
tls_key_file  = "C:/MessageFoundry/tls/engine-key.pem"
```

The configuration above also requires a revocation attestation. The engine does not check certificate revocation through OCSP or CRL. It refuses an off-loopback bind with in-process TLS (**exit 2**) until you attest to your revocation controls. Set the attestation as an **environment variable, not a TOML key**:

```
setx MEFOR_TLS_REVOCATION_ATTESTED 1
```

For NSSM, set it on the service: `nssm set MessageFoundry AppEnvironmentExtra MEFOR_TLS_REVOCATION_ATTESTED=1`. This confirms that you accept responsibility for revocation checking (ADR 0078).

Notes:

- `listen_address = "0.0.0.0"` listens on all network interfaces. You can instead use a specific address (e.g. `"10.0.0.12"`) to limit it to one network.
- `serve_web_console` must be set **explicitly** when the engine is exposed. The console defaults on only for loopback binds. An exposed instance serves JSON alone unless the console is explicitly enabled.
- If the private key requires a password, set `MEFOR_API_TLS_KEY_PASSWORD` to its passphrase. Never put the passphrase in the file.
- The engine **will refuse to start** if you open it to the network **without** a certificate (this protects you from accidentally sending credentials in clear text).

Open the firewall for the selected port, which defaults to **8765/TCP**. Then restart the MessageFoundry service.

Reverse proxy or load balancer: A proxy can terminate TLS before traffic reaches the engine. In `[api]`, set `tls_terminated_upstream = true`. Add the proxy address to `trusted_proxies = ["..."]`. Configure the proxy to forward requests to the engine. Refer to `REMOTE-CONSOLE.md` for details.

Step 3 — Connect the console from a remote PC

For the current web console, open `https://<engine-host>:8765/ui` in a browser. The commands below apply only to the retired desktop console and remain as historical reference:

```
messagefoundry-console --url https://mefor-srv01.hospital.local:8765
```

(or, from a command prompt, `python -m messagefoundry.console --url https://mefor-srv01.hospital.local:8765`)

Certificate trust for the retired desktop client:

- A domain-joined PC already trusts a certificate issued by its organization's CA.
- If you used a **public CA**, it is also already trusted.
- If you used a **self-signed certificate** (or an internal CA not yet installed on the PC), add `--cacert` pointing at the engine's certificate file:

```
messagefoundry-console --url https://mefor-srv01.hospital.local:8765 --cacert  
C:\MessageFoundry\tls\engine-cert.pem
```

(Alternatively, install your internal CA into the PC's Windows certificate store once, and you can drop the flag.)

Finally, **sign in** with your MessageFoundry account. If multi-factor authentication is enabled, enter the second factor when prompted. This follows the local-console sign-in process.

Step 4 — Verify it's working

- The console connects and shows the engine **Status** page with live connection counts.
- The status indicator shows the engine is reachable and refreshes on its own.

If the console cannot connect, see Troubleshooting below.

Optional — require a client certificate (mutual TLS)

You can require a client certificate as well as user sign-in. Only PCs with an approved certificate can connect. The client flags below apply to the retired desktop console.

- On the engine, set `tls_client_ca_file` under `[api]` to the CA that issued the console certificates.
- On each console, add `--client-cert` (and `--client-key` if the key is separate):

```
messagefoundry-console --url https://mefor-srv01.hospital.local:8765 ^ --client-cert  
C:\MessageFoundry\tls\console.pem --client-key C:\MessageFoundry\tls\console-key.pem
```

This is optional and off by default.

Troubleshooting

What you see	What it means / what to do
<code>certificate verify failed</code> / "not trusted by the trust provider"	The PC does not trust the engine's certificate. Add <code>--cacert <file></code> , or install the issuing CA into the PC's Windows certificate store.
<code>refusing to use plaintext</code> <code>http to non-loopback host</code> ...	You used an <code>http://</code> address to a remote engine. Use the <code>https://</code> address (configure the certificate in Step 2).
The engine will not start after editing the config	Check the certificate and revocation attestation. Set <code>tls_cert_file</code> under <code>[api]</code> . If the key is separate, also set <code>tls_key_file</code> . Set <code>MEFOR_TLS_REVOCATION_ATTESTED=1</code> (Step 2). To restore local-only access, set <code>[security].local_access_only = true</code> .
<code>moved to [security]. ...</code> and <code>is no longer accepted</code>	You used a pre-ADR-0118 key such as <code>[api].host</code> . Exposure now lives in <code>[security]</code> (<code>local_access_only</code> / <code>listen_address</code>). The old spellings are rejected when the config loads.
"hostname mismatch" when connecting	The certificate's name (SAN) does not match the address in <code>--url</code> . Reissue the certificate for the correct hostname/IP.

What you see	What it means / what to do
Console cannot reach the server at all	Check the firewall on the engine server (default port 8765/TCP) and that the service is running.

Security notes

- **Stays on your network.** The engine runs on-premises. Remote access is to *your* server over *your* network — no data leaves your environment.
- **Encrypted in transit.** All console–engine traffic, including login, is protected by TLS.
- **Authenticated and audited.** Every user signs in, and access to patient data is recorded with the acting user. We recommend enabling **multi-factor authentication** for any network-exposed engine.
- `--insecure` only permits unencrypted `http` on a trusted test network. It does **not** weaken certificate checking for `https`. Do not use it for production.

More information

- **Technical reference** (full `[api]` settings, in-process vs. Upstream TLS): `docs/REMOTE-CONSOLE.md`
- **Security overview** (authentication, TLS, auditing): `docs/SECURITY.md`
- **Running the engine as a service:** `docs/SERVICE.md`

For rollout questions, use the project’s contact page.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.